

CookVerse — Application Documentation

Version: 1.0.0
Date: July 2026
Platform: Mobile (iOS / Android via Expo Go) + REST API Backend

1. Executive Summary

CookVerse is an AI-powered social recipe platform. Users can discover recipes in a TikTok-style feed, browse categories, upload their own recipes, search with AI fallback, generate recipes from dish names or ingredients, like/save/comment, and follow other chefs.

The project is split into two main parts:

Layer	Location	Purpose
Backend	backend/	REST API, database, AI recipe generation
Mobile	mobile/	React Native app (Expo SDK 54)

2. Technology Stack

2.1 Frontend (Mobile)

Technology	Version	Purpose
React Native	0.81.5	Cross-platform mobile UI
Expo SDK	54	Build tooling, native modules, Expo Go
Expo Router	6	File-based navigation
Clerk Expo	2.13	Authentication (email/password, Google OAuth)
Expo Image	3.x	Optimized image loading
Expo Image Picker	17.x	Photo upload for recipes
React Native Reanimated	4.x	Animations
Expo Linear Gradient	15.x	UI gradients on feed & detail screens
Expo Secure Store	15.x	Secure token storage (via Clerk)
Expo Web Browser	15.x	OAuth flows

2.2 Backend

Technology	Version	Purpose
Node.js	22+	Runtime
Express	5.1	HTTP API server

Drizzle ORM	0.44	Type-safe database queries
Neon PostgreSQL	Serverless	Cloud PostgreSQL database
OpenAI API	gpt-4o-mini	AI recipe generation
CORS	2.8	Cross-origin requests from mobile
dotenv	16.x	Environment variables
Nodemon	3.x	Dev hot-reload

2.3 External Services & APIs

Service	Used For
Clerk	User authentication, sessions, OAuth
Neon PostgreSQL	Persistent data storage
OpenAI	AI recipe generation (ingredients, steps, images)
Unsplash	Recipe & food images (URLs)
DiceBear	Default avatar placeholders

3. Project Structure

```
Recepie/
├── backend/
│   ├── src/
│   │   ├── config/           # env, db, cron
│   │   ├── data/             # seedRecipes.js (category recipes)
│   │   ├── db/               # schema.js, migrations
│   │   ├── routes/           # users, recipes, follows
│   │   ├── services/         # aiRecipe.js (OpenAI)
│   │   ├── utils/            # format, recipeImages, spellCorrect, seedRepair
│   │   └── server.js          # Entry point
│   ├── .env                  # DATABASE_URL, OPENAI_API_KEY, PORT
│   └── package.json
├── mobile/
│   ├── app/                  # Expo Router screens
│   │   ├── (auth)/           # splash, welcome, sign-in, sign-up, forgot-password
│   │   ├── (tabs)/           # home, categories, search, generate, upload, profile
│   │   ├── create-profile.jsx
│   │   └── recipe/[id].jsx
│   ├── components/           # FeedPost, CookVerseLogo, AIRecipeCard, etc.
│   ├── constants/            # api, colors, categories, routes
│   ├── services/             # cookverseAPI.js
│   ├── assets/styles/        # Shared StyleSheets
│   └── .env                   # EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY
```

```
|   └─ package.json
|
└─ docs/
    └─ CookVerse-Documentation.md (this file)
```

4. Environment Variables

Backend (backend/.env)

Variable	Required	Description
DATABASE_URL	Yes	Neon PostgreSQL connection string
OPENAI_API_KEY	Yes*	OpenAI API key for AI recipes (*falls back to mock if missing)
PORT	No	Server port (default: 5001)
NODE_ENV	No	production enables cron jobs

Mobile (mobile/.env)

Variable	Required	Description
EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY	Yes	Clerk publishable key for auth

API URL (Mobile)

The mobile app auto-detects the dev machine IP from Expo for physical devices:

```
http://<your-computer-ip>:5001/api
```

Defined in `mobile/constants/api.js` .

5. Database Schema (PostgreSQL)

Database: **Neon PostgreSQL**

ORM: **Drizzle**

Tables

users

Column	Type	Description
id	text (PK)	Clerk user ID
full_name	text	Display name
age	integer	Optional
food_preference	text	Veg / Non-Veg / etc.
diet_goal	text	Gym, Weight Loss, etc.

bio	text	Profile bio
avatar_url	text	Profile image URL
created_at	timestamp	Registration time

recipes

Column	Type	Description
id	serial (PK)	Recipe ID
title	text	Recipe name
image	text	Image URL
video_url	text	Optional YouTube URL
ingredients	text (JSON)	Array of {name, quantity}
steps	text (JSON)	Array of instruction strings
category	text	e.g. Gym Recipes, Bengali
cuisine	text	Cuisine type
diet_type	text	Vegetarian, Non-Veg, Vegan
calories	integer	Calorie count
nutrition	text (JSON)	protein, carbs, fat
difficulty	text	Easy / Medium / Hard
cooking_time	text	e.g. "45 mins"
tips	text	Chef tips
is_ai_generated	boolean	AI vs user upload
created_by	text	Clerk user ID
creator_name	text	Display name
creator_avatar	text	Avatar URL
likes_count	integer	Like counter
created_at	timestamp	Post time

comments

Recipe comments with userId, userName, userAvatar, text.

likes

Unique per (recipe_id, user_id) — toggle like/unlike.

saves

Unique per (recipe_id, user_id) — bookmark recipes.

follows

Unique per (follower_id, following_id) — follow chefs.

ai_history

Stores JSON recipe data from AI generations per user.

favorites (legacy)

Older favorites table; still supported via `/api/favorites` endpoints.

6. Backend API Reference

Base URL: `http://localhost:5001/api` (development)

Health

Method	Endpoint	Description
GET	<code>/health</code>	Server health check

Users (`/api/users`)

Method	Endpoint	Description
GET	<code>/search/:query</code>	Search users by name
GET	<code>/:userId</code>	Get user profile + stats
POST	<code>/</code>	Create or update profile

Recipes (`/api/recipes`)

Method	Endpoint	Description
GET	<code>/feed</code>	Paginated feed (mode: latest, trending, foryou, following)
GET	<code>/search</code>	Search recipes, hashtags
GET	<code>/category/:name</code>	Recipes by category or cuisine
GET	<code>/user/:userId</code>	User's posts, saved, or liked recipes
GET	<code>/:id</code>	Single recipe detail
POST	<code>/</code>	Create new recipe
POST	<code>/ai/generate</code>	Generate recipe via AI
GET	<code>/ai/history/:userId</code>	AI generation history

GET	<code>/:id/comments</code>	Get comments
POST	<code>/:id/comments</code>	Add comment
POST	<code>/:id/like</code>	Toggle like
GET	<code>/:id/like/:userId</code>	Like status
POST	<code>/:id/save</code>	Toggle save
GET	<code>/:id/save/:userId</code>	Save status

Follows (`/api/follows`)

Method	Endpoint	Description
POST	<code>/toggle</code>	Follow / unfollow user
GET	<code>/status/:followerId/:followingId</code>	Follow status
GET	<code>/followers/:userId</code>	List followers
GET	<code>/following/:userId</code>	List following

AI Generate Request Body

```
{
  "query": "how to cook biryani",
  "dishName": "biryani",
  "ingredients": ["egg", "rice", "onion"],
  "userId": "clerk_user_id"
}
```

Feed Query Parameters

Param	Values	Description
mode	latest, trending, foryou, following	Feed algorithm
page	number	Page number (default 1)
limit	number	Items per page (default 8-10)
userId	string	Required for following mode
category	string	Optional category filter

7. AI Recipe Service

File: `backend/src/services/aiRecipe.js`

Features

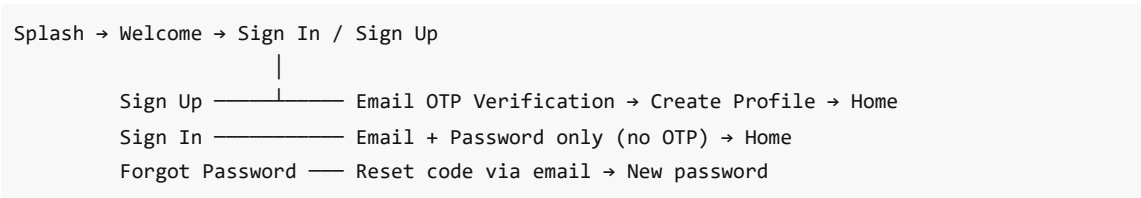
- Generate recipes from **dish name** (e.g. "how to cook biryani")
- Generate recipes from **ingredients** (e.g. egg, rice, onion)
- **Spell correction** for typos (biryani → biryani, chicken → chicken)
- Returns structured JSON: title, ingredients, steps, cookingTime, difficulty, calories, nutrition, tips, image
- **Image matching** via `recipeImages.js` (Unsplash URLs by dish keyword)
- Falls back to detailed mock recipes if OpenAI key is missing or API fails
- Saves to `ai_history` table when userId provided

OpenAI Model

- **Model:** `gpt-4o-mini`
- **Max tokens:** 2500
- **Response format:** JSON object

8. Mobile App — User Flow

8.1 Authentication Flow



- **Sign Up:** Email verification code required (OTP)
- **Sign In:** Email + password only — no verification code for existing users
- **Clerk** handles sessions; tokens stored in Expo Secure Store

8.2 Onboarding

After first sign-up, user completes **Create Profile** (name, preferences, diet goals) before accessing the main app.

8.3 Main Navigation (Bottom Tabs)

Tab	Screen	Description
Home	<code>index.jsx</code>	TikTok-style recipe feed
Categories	<code>categories.jsx</code>	Browse recipes by category
AI Chef	<code>generate.jsx</code>	Center floating button — AI recipe generator
Upload	<code>upload.jsx</code>	Post new recipe with photo
Profile	<code>profile.jsx</code>	User profile, recipes, saved, liked

Hidden routes: Search (accessible from Home header), Favorites

8.4 Home Feed

- **Latest** — newest uploads first
- **Trending** — most liked recipes
- **For You** — mix of popular + recent
- **Following** — recipes from followed chefs

- Vertical swipe (paging) through full-screen recipe cards
- Actions: Like, Comment, Share, Save, Follow creator

8.5 Categories

- **People-Based:** Gym, Weight Loss, Diabetic Friendly, High Protein, etc.
- **Cuisines:** North Indian, South Indian, Bengali, Italian, etc.
- Each category lists recipes with images; tap for full detail

8.6 Search

- Search recipes, users, hashtags
- Filter tabs: All, Recipes, Users, Tags
- If no results found → **AI auto-generates** recipe with image

8.7 AI Generator

- **By Dish:** "How to cook biryani"
- **By Ingredients:** Add chips (egg, rice, onion) → generate
- Shows full recipe with image, ingredients, step-by-step instructions
- Option to **Share to Feed**

8.8 Upload Recipe

- Photo (image picker), title, ingredients (one per line), steps (one per line)
- Category and diet type selection
- Posts to feed via API

8.9 Recipe Detail

- Full image, stats (time, calories, difficulty)
- Ingredients list with quantities
- Step-by-step instructions
- Like, save, comment, follow creator
- Nutrition info (protein, carbs, fat)

9. Recipe Categories

People-Based

Gym Recipes, Weight Loss, Diabetic Friendly, Patient Food, High Protein, Kids Recipes, Vegan, Vegetarian

Cuisines

North Indian, South Indian, Bengali, Punjabi, Gujarati, Rajasthani, Chinese, Italian, Street Food

10. Seed Data

On server startup:

1. `seedRecipesIfEmpty()` — inserts category recipes if titles don't exist
2. `repairBrokenRecipes()` — fixes recipes with missing ingredients/steps

~35+ sample recipes across all categories with images, ingredients, and steps.

11. How to Run the Application

Prerequisites

- Node.js 18+
- npm
- Expo Go app on phone (SDK 54)
- Clerk account with publishable key
- Neon PostgreSQL database
- OpenAI API key (optional but recommended)

Backend

```
cd backend
npm install
# Configure backend/.env
npm run dev
```

Server runs at: `http://localhost:5001`

Mobile

```
cd mobile
npm install
# Configure mobile/.env with EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY
npx expo start --clear
```

Scan QR code with Expo Go on the same Wi-Fi network.

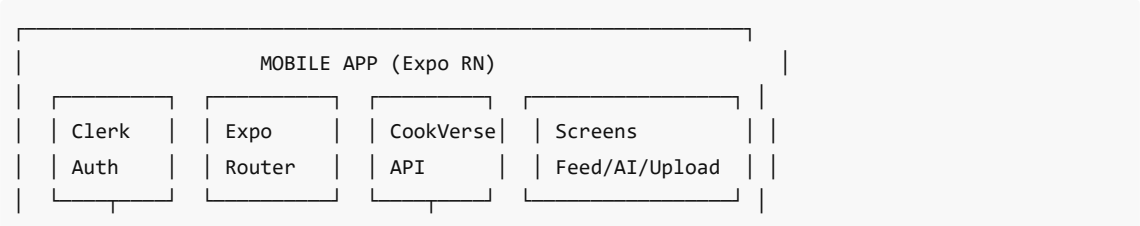
Database Migration

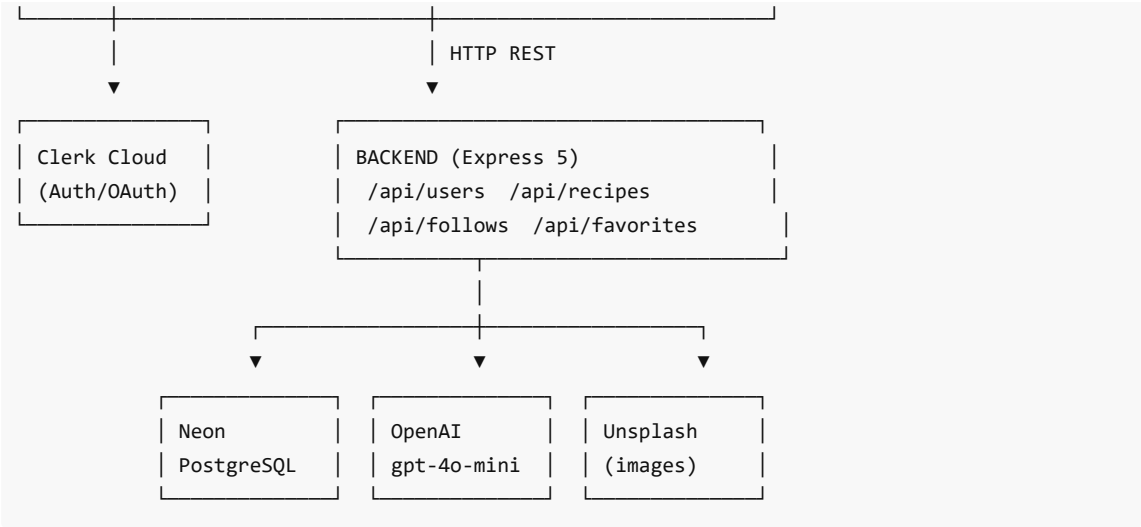
SQL migration file: `backend/src/db/migrations/0001_cookverse.sql`
Or use: `npx drizzle-kit push` from backend directory.

12. Security Notes

- Clerk handles authentication; backend uses Clerk user IDs
- No JWT validation on backend API currently — relies on client passing correct `userId`
- `.env` files must not be committed to git
- CORS enabled for development

13. Architecture Diagram





14. Key Files Reference

File	Purpose
backend/src/server.js	API entry, seeding, routes mount
backend/src/services/aiRecipe.js	AI recipe generation
backend/src/db/schema.js	Database table definitions
mobile/services/cookverseAPI.js	All API calls from mobile
mobile/app/(tabs)/index.jsx	Home feed screen
mobile/app/(tabs)/generate.jsx	AI generator screen
mobile/components/FeedPost.jsx	TikTok-style feed card
mobile/constants/routes.js	Route path constants

15. Future Enhancements (Suggested)

- Backend JWT validation with Clerk
- Cloudinary for image uploads
- Push notifications
- Video recipe support
- In-app messaging
- Admin dashboard